SOP: Azure VM Image Capture with Compute Gallery (Nginx)

Oct 30, 2026 · @ashutosh

Overview

This SOP captures a running Nginx VM into an Azure Compute Gallery, rolls out three image versions (1.0.0 → 2.0.0 → 3.0.0), keeps only the newest marked as **latest**, and ends with a CLI command that always builds a VM from that latest version.

Scope: Linux VM (Ubuntu) serving `/var/www/html/index.html` through Nginx, all steps in the Azure Portal (portal.azure.com), plus one Azure CLI command at the end to build a VM from the latest image.

How "latest" works in Azure: when a VM is created from an image definition without a version (or with `/versions/latest`), Azure picks the **highest version number** whose `excludeFromLatest` flag is `false`. This SOP sets that flag (the Exclude from latest checkbox in the portal) on every version so the newest is always the one picked and older ones are never picked by accident.

Why Specialized images: a Specialized image is captured from a running VM without deprovisioning it. The source VM keeps serving traffic, and every VM built from the image boots with the same Nginx config and content. A Generalized image would require `waagent -deprovision` and `az vm generalize`, which permanently retires the source VM.

Version	Built from	index.html shows	excludeFromLatest at the end
1.0.0	Original VM `vm-nginx-src`	Version 1 page	true
2.0.0	`vm-web-v1` (edited)	Version 2 page	true
3.0.0	`vm-web-v2` (edited)	Version 3 page	**false (LATEST)**

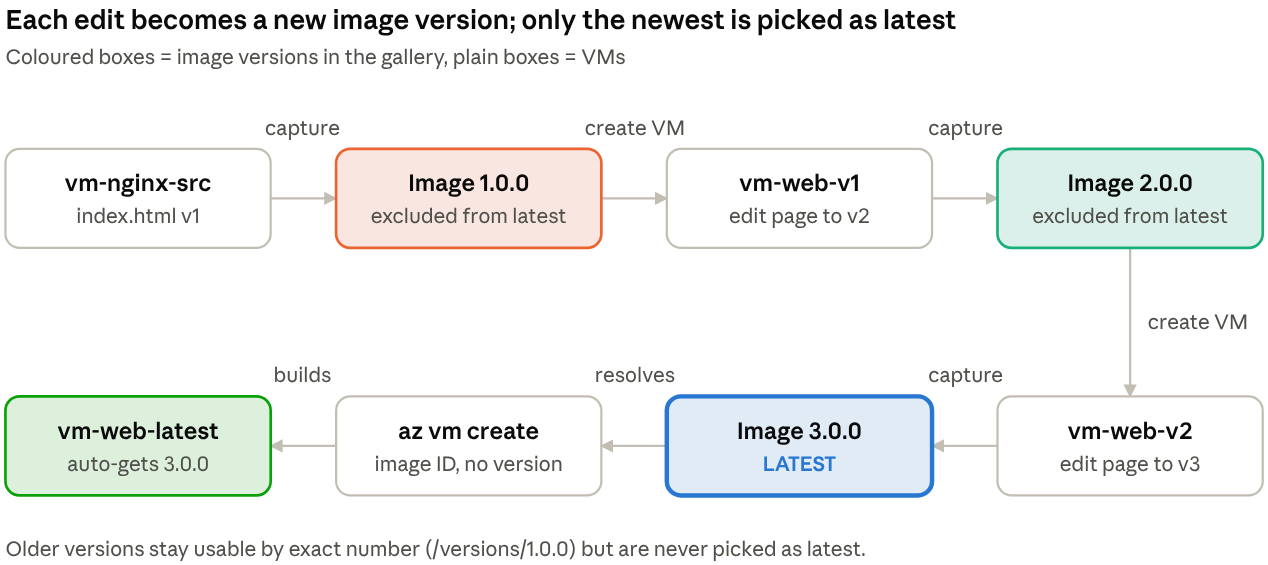


image lifecycle · 3 versions, 1 latest

Each round builds a VM from the previous version, edits the page, and captures the next version; the final `az vm create` always lands on the newest.

Step 0 · Prerequisites and naming

Use these names throughout (change them to your own), and check the source VM before you start.

Item	Value used in this SOP
Resource group	`rg-nginx-images`
Region	Central India (same as the source VM)
Source VM	`vm-nginx-src`
Compute Gallery	`galNginxImages`
Image definition	`nginx-web`
Image versions	`1.0.0`, `2.0.0`, `3.0.0`

- [ ] Signed in to **portal.azure.com** with Contributor rights on the resource group
- [ ] Source VM is **Running** and `http://<public-IP>` shows the Nginx page

- [ ] Note the VM's **VM generation** and **Security type**: open the VM → **Overview** → **Properties** tab (shown as e.g. *V2* and *Trusted launch*). The capture wizard copies these automatically.
- [ ] Inbound port **80** (HTTP) and **22** (SSH) allowed in the VM's network security group

Step 1 · Create the Azure Compute Gallery

The gallery is the container that will hold the image definition and all its versions.

1. In the portal search bar, type **Azure compute galleries** and open it.
2. Click **+ Create**.
3. **Basics** tab:
   - **Subscription:** your subscription
   - **Resource group:** `rg-nginx-images`
   - **Name:** `galNginxImages` (letters, numbers, dots and underscores only; no hyphens)
   - **Region:** Central India
   - **Description:** Nginx web server golden images
4. **Sharing method** tab: keep **Role based access control (RBAC)**.
5. Click **Review + create** → **Create**.
6. When the deployment finishes, click **Go to resource**.

> **Checkpoint:** the gallery's Overview page opens and shows no image definitions yet. The definition is created in the next step, directly from the capture wizard.

Step 2 · Capture image version 1.0.0 from the source VM

The capture wizard creates the image definition and version 1.0.0 in one pass, while the source VM keeps running.

1. Go to **Virtual machines** → open `vm-nginx-src`.
2. On the **Overview** toolbar, click **Capture** (in newer portals: **Capture** → **Image**).
3. **Basics** tab, Project details:
   - **Resource group:** `rg-nginx-images`
   - **Region:** Central India

4. **Share image to Azure compute gallery:** select **Yes, share it to a gallery as a VM image version**.
5. **Target Azure compute gallery:** select `galNginxImages`.
6. **Operating system state:** select **Specialized** (the VM is not deprovisioned and keeps running).
7. **Target VM image definition:** click **Create new** and fill in:
   - **VM image definition name:** `nginx-web`
   - **Publisher:** `MyOrg` · **Offer:** `NginxWeb` · **SKU:** `Ubuntu-Nginx`
   - Click **OK** (VM generation and security type are copied from the VM).
8. **Version details:**
   - **Version number:** `1.0.0`
   - **Exclude from latest:** leave **unchecked** (1.0.0 is the only version, so it is latest)
   - **End of life date:** optional
9. **Replication: Default replica count** `1`, target region Central India.
10. **Storage type:** Standard HDD LRS (cheapest for labs) or Premium SSD LRS for production.
11. Click **Review + create** → **Create**.

> **Tip:** capture takes 10 to 30 minutes. Pause writes on the VM during capture for a consistent snapshot.

> **Checkpoint:** Compute gallery → `nginx-web` → **Versions** tab shows `1.0.0` with Provisioning state **Succeeded**.

Step 3 · Round 1: VM from 1.0.0 → edit index.html → version 2.0.0 (latest)

Build a VM from 1.0.0, change the page, capture it as 2.0.0, then switch 1.0.0 out of "latest".

3.1 Create a VM from version 1.0.0

1. Go to **Azure compute galleries** → `galNginxImages` → `nginx-web` → **Versions** tab → click **1.0.0**.
2. On the version's toolbar, click **+ Create VM**.

3. **Basics** tab:
   - **Resource group:** `rg-nginx-images` · **Virtual machine name:** `vm-web-v1`
   - **Region:** Central India · **Size:** Standard_B1s
   - **Image:** already set to `nginx-web - 1.0.0`
   - **Administrator account:** greyed out (a Specialized image keeps the original user and SSH key)
   - **Public inbound ports:** Allow selected ports → **HTTP (80)** and **SSH (22)**
4. Click **Review + create** → **Create**.
5. Open the new VM, copy its **Public IP address**, browse to it: you should see the Version 1 page.

3.2 Update index.html to Version 2

1. In `vm-web-v1`, go to **Operations** → **Run command** → **RunShellScript**.
2. Paste the script below and click **Run** (output appears in about 30 seconds).

```
cat <<EOF | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><head><title>Nginx Image v2</title></head>
<body style="font-family:Arial;background:#0f766e;color:#fff;text-
align:center;padding-top:80px">
  <h1>Version 2.0.0</h1>
  <p>Served from image nginx-web 2.0.0</p>
</body></html>
EOF
sudo nginx -t && sudo systemctl reload nginx
```

3. Refresh `http://<vm-web-v1 public IP>`: the page is now teal and reads **Version 2.0.0**.

> Prefer SSH? Click **Connect** → **SSH**, then run `sudo nano /var/www/html/index.html`, save, and `sudo systemctl reload nginx`.

3.3 Capture version 2.0.0 and keep it as latest

1. In `vm-web-v1`, click **Capture** on the Overview toolbar.
2. Fill in as in Step 2, with these differences:
   - **Target Azure compute gallery:** `galNginxImages`

- **Operating system state:** Specialized
- **Target VM image definition:** select the existing **`nginx-web`** (do not create new)
- **Version number:** `2.0.0`
- **Exclude from latest: unchecked**

3. Click **Review + create** → **Create** and wait for **Succeeded**.

3.4 Remove "latest" from version 1.0.0

1. Go to `galNginxImages` → `nginx-web` → **Versions** → click **1.0.0**.
2. Under **Settings**, open **Configuration**.
3. Tick **Exclude from latest** → click **Save**.

> **Result:** 2.0.0 is latest. 1.0.0 remains usable when chosen by exact version, but is never picked automatically.

Step 4 · Round 2: VM from 2.0.0 → edit index.html → version 3.0.0 (latest)

Same cycle once more; 3.0.0 becomes latest and 2.0.0 is switched out.

4.1 Create a VM from version 2.0.0

1. `galNginxImages` → `nginx-web` → **Versions** → click **2.0.0** → **+ Create VM**.
2. **Virtual machine name:** `vm-web-v2`; everything else as in 3.1 (Standard_B1s, ports 80 and 22).
3. **Review + create** → **Create**. Browse to its public IP: you should see the teal Version 2 page.

4.2 Update index.html to Version 3

In `vm-web-v2` → **Operations** → **Run command** → **RunShellScript**, paste and **Run**:

```
cat <<EOF | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><head><title>Nginx Image v3</title></head>
<body style="font-family:Arial;background:#7c3aed;color:#fff;text-
align:center;padding-top:80px">
  <h1>Version 3.0.0</h1>
  <p>Served from image nginx-web 3.0.0 (LATEST)</p>
</body></html>
EOF
sudo nginx -t && sudo systemctl reload nginx
```

Refresh the browser: the page is now purple and reads **Version 3.0.0**.

4.3 Capture version 3.0.0 and keep it as latest

1. In `vm-web-v2`, click **Capture**.
2. Gallery `galNginxImages` · OS state **Specialized** · existing definition **`nginx-web`** · **Version number** `3.0.0` · **Exclude from latest unchecked**.
3. **Review + create** → **Create** and wait for **Succeeded**.

4.4 Remove "latest" from version 2.0.0

1. `nginx-web` → **Versions** → click **2.0.0** → **Settings** → **Configuration**.
2. Tick **Exclude from latest** → **Save**.

> **Result:** 3.0.0 is latest; 1.0.0 and 2.0.0 are both excluded.

Step 5 · Verify which version is latest

Only 3.0.0 should have **Exclude from latest = No**.

1. Go to `galNginxImages` → `nginx-web` → **Versions** tab.
2. If the column is hidden, click **Edit columns** and add **Exclude from latest**.
3. Compare with the expected view:

Version	Exclude from latest	Provisioning state	Role
3.0.0	No	Succeeded	**LATEST** (default pick)

Version	Exclude from latest	Provisioning state	Role
2.0.0	Yes	Succeeded	Previous, by exact version only
1.0.0	Yes	Succeeded	Original, by exact version only

> If two versions show **No**, Azure picks the higher number, but set the older one to **Yes** anyway so the gallery state is unambiguous.

Step 6 · Create a VM that always uses the LATEST image

Point at the image **definition**, not a version; Azure resolves it to the current latest (3.0.0 today, 4.0.0 after the next round) with no change to the command.

Option A · CLI command (auto-picks latest)

Run in **Azure Cloud Shell** (the `>_` icon in the portal top bar, choose **Bash**):

```
IMG_DEF_ID=$(az sig image-definition show \
  --resource-group rg-nginx-images \
  --gallery-name galNginxImages \
  --gallery-image-definition nginx-web \
  --query id -o tsv)

az vm create \
  --resource-group rg-nginx-images \
  --name vm-web-latest \
  --image "$IMG_DEF_ID" \
  --specialized \
  --security-type TrustedLaunch \
  --size Standard_B1s \
  --public-ip-sku Standard

az vm open-port -g rg-nginx-images -n vm-web-latest --port 80 --priority 1010

# Confirm which version it used (expect 3.0.0)
az vm show -g rg-nginx-images -n vm-web-latest \
  --query storageProfile.imageReference.exactVersion -o tsv
```

> Drop `--security-type TrustedLaunch` if the source VM's Security type was **Standard**. `--image "$IMG_DEF_ID/versions/latest"` works the same way.

Option B · Portal (also picks latest)

1. Go to `galNginxImages` → open the **`nginx-web`** definition (not a version).
2. Click **+ Create VM** on the definition's toolbar; the Image field reads `nginx-web - latest`.
3. Name it, allow ports 80 and 22, **Review + create** → **Create**.
4. Check the result: VM → **Overview** → **Properties** → **Image** shows the exact version used (3.0.0).

Troubleshooting

Symptom	Likely cause	Fix in the portal
Capture fails with a VM generation mismatch	Existing definition was made for a different generation	Create a new definition from the capture wizard (**Create new**)
Capture fails with a security type error	Definition lacks Trusted launch	Create a new definition; the wizard copies security type from the VM
Create VM asks for admin username/password	Definition was set to Generalized	Recreate the definition with OS state **Specialized**
New VM shows an old page	An older version still has Exclude from latest = No	Version → **Configuration** → tick **Exclude from latest** → Save
Page not reachable on new VM	Port 80 not open	VM → **Networking** → **Add inbound port rule** → HTTP 80
Capture stuck for a long time	Large disk or many replicas	Wait; keep replica count 1 for labs

Rollback and cleanup

Roll back latest to 2.0.0 (nothing deleted):

1. Version **3.0.0** → **Configuration** → tick **Exclude from latest** → **Save**.
2. Version **2.0.0** → **Configuration** → untick **Exclude from latest** → **Save**.

Delete lab resources (order matters: VMs, then versions, then definition, then gallery):

1. **Virtual machines** → select `vm-web-v1`, `vm-web-v2`, `vm-web-latest` → **Delete**; tick **Delete with VM** for OS disk, NIC and public IP.
2. `galNginxImages` → `nginx-web` → **Versions** → delete 3.0.0, 2.0.0, 1.0.0.
3. `galNginxImages` → select `nginx-web` → **Delete**.
4. Open `galNginxImages` → **Delete**.

> If `rg-nginx-images` holds only lab resources, deleting the resource group removes everything in one go.